Büyük Veri İşleme ve Yönetimi

Hafta ….

A Comparative Analysis of Big Data Frameworks:
An Adoption Perspective (Büyük Veri Çerçevelerinin Karşılaştırmalı Analizi:
Uygulama Açısından Bir Bakış)

- https://www.mdpi.com/2076-3417/11/22/11033

Makalenin Özeti

- Bu makalede, açık kaynaklı ve yaygın kullanılan büyük veri işleme çerçeveleri çeşitli temel özellikler üzerinden karşılaştırılmaktadır. Karşılaştırma için ilişkili özellikler yedi farklı özellik vektöründe gruplandırılmış ve çerçevelerin güçlü ve zayıf yönleri incelenmiştir. Çalışmanın amacı, farklı kullanım senaryolarına göre uygun büyük veri işleme çalışma alanının (Framework) seçilmesine yardımcı olacak bir karar rehberi sunmaktır.

Üretilen veri miktarı

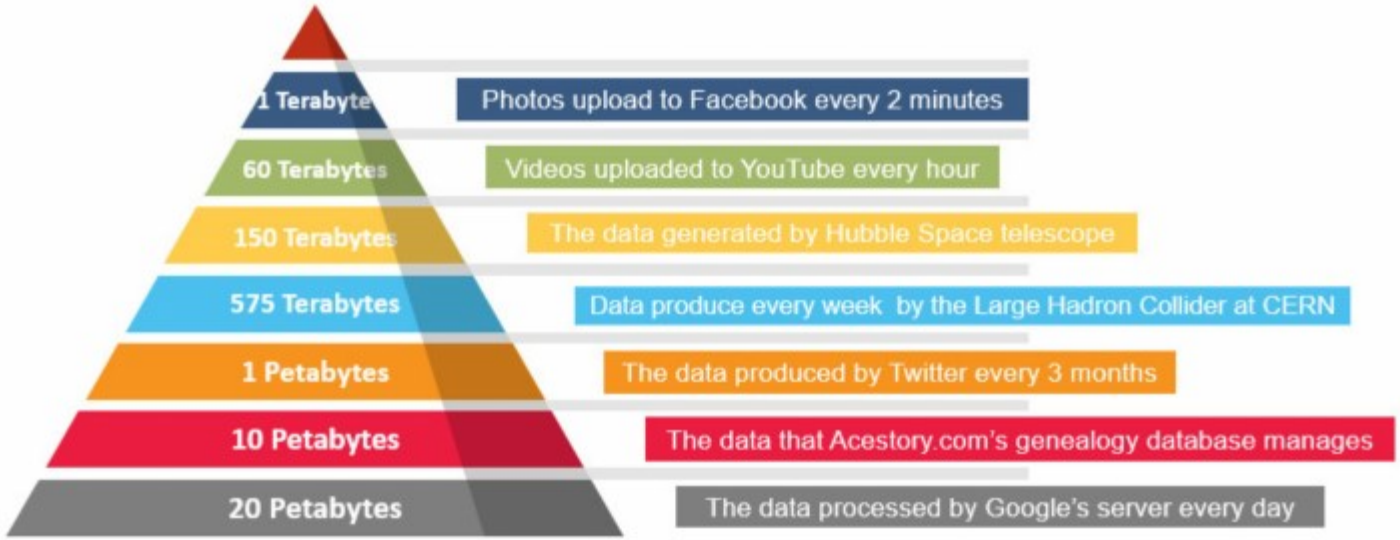


Figure 1. Understanding the data deluge.

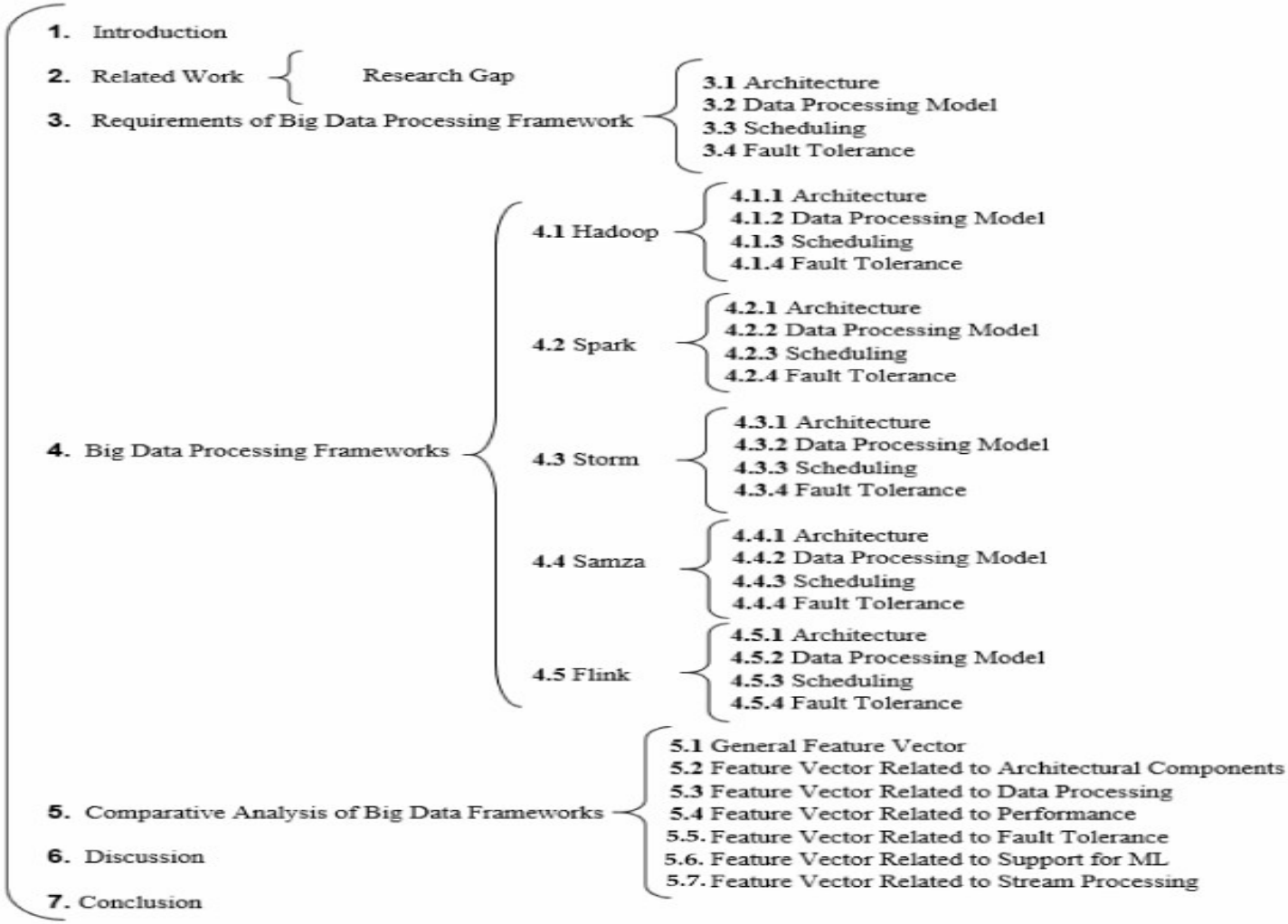


Figure 2. Organization of the paper.



3. Requirements of a Big Data Processing Framework (Büyük Veri İşlemenin Çalışma Alanı Gereksinimleri)

- 3.1. Architecture (Mimarisi)
- 3.2. Data Processing Model (Veri İşleme Modeli)
- 3.3. Scheduling (Zamanlama)
- 3.4. Fault Tolerance (Hata Toleransı)

• 3.1. Architecture (Mimarisi)

- Büyük veri çalışma alanı mimarilerinin çoğu, dört ana modül içerir: kaynak yönetimi, zamanlama, yürütme ve depolama.
- Mimari, veri kaynağından başlayarak gelecekte kullanılmak üzere sonuçların oluşturulması ve depolanmasına kadar tüm veri yaşam döngüsünü ana hatlarıyla belirtmelidir. Bu mimari, hata toleransı, ölçeklenebilirlik ve yüksek kullanılabilirliği sağlamalıdır.

3.2. Data Processing Model (Veri İşleme Modeli)

- Veri işleme modeli, verilerin nasıl işlendiğini ve hesaplamaların sistemde nasıl temsil edildiğini tanımlar. Genel olarak veriler ya toplu iş modunda ya da sürekli bir akış şeklinde işlenir. Toplu işleme sisteminde işleme, belirli bir süre boyunca depolanan bir veri bloğu üzerinde gerçekleştirilir. Oysa akış işleme sisteminde ise işleme, gelen verilerin sürekli akışı üzerinde gerçekleştirilir.
- Hesaplamalar da farklı biçimlerde temsil edilir; örneğin, Hadoop hesaplamaları map-reduce işlevlerinde işler. Alternatif olarak, veriler ve hesaplamalar bir mantıksal yönlü döngüsüz grafik (DAG) olarak da gerçekleştirilebilir.

3.3. Scheduling (Zamanlama)

- Görev zamanlamasının temel amacı, bağımsız ve bağımlı görevleri zamanlamak ve görev geçişlerinin sayısını en uygun şekilde azaltarak, hesaplama süresini kısaltırken kaynak kullanımını artırmaktır.

3.4. Fault Tolerance (Hata Toleransı)

- Hata toleransı, bir veya daha fazla bileşen arızalandığında sistemin çalışmaya devam etmesini sağlayan bir özelliktir.

4. Big Data Processing Frameworks (Büyük Veri İşleme Çalışma Alanları)

- Büyük veri kümelerinin işlenmesi için paralel ve dağıtık hesaplama yöntemleri geliştirilmiştir. Bu amaçla veri dağıtımı, ölçeklenebilirlik ve hata toleransı gibi sorunları yöneten çeşitli veri işleme çerçeveleri ortaya çıkmıştır. MapReduce, toplu veri işleme için geliştirilen öncü modellerden biri olmuş ve Apache Hadoop ile yaygınlaşmıştır. Akış verilerinin işlenmesi için ise Spark, Storm, Samza ve Flink gibi yeni çalışma alanları geliştirilmiştir. Makale, bu alandaki en yaygın beş çalışma alanını karşılaştırmaktadır.

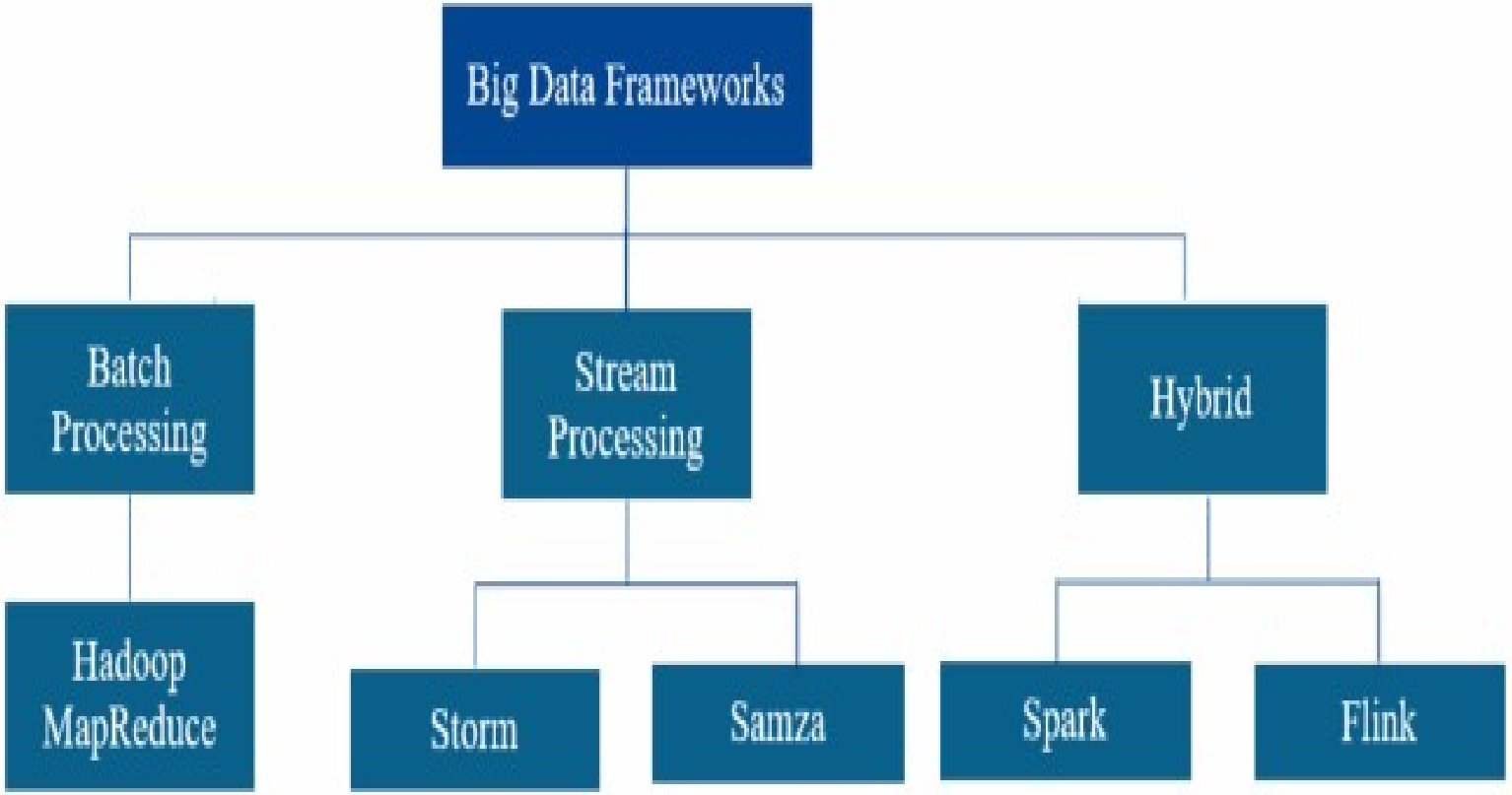


Figure 4. Classification of big data processing frameworks.

4.1.1. Architecture (Hadoop Mimarisi)

- Hadoop mimarisi iki temel bileşenden oluşur: HDFS ve YARN. HDFS, verileri bloklara bölerek farklı düğümlerde saklar ve blokları çoğaltarak hata toleransı ve yüksek erişilebilirlik sağlar. HDFS'de NameNode dosya sistemini yönetirken, DataNode veri bloklarını depolar ve veri erişimini sağlar. YARN ise kümedeki kaynak yönetimi ve görev zamanlamasını gerçekleştirir; Resource Manager genel kaynakları yönetirken, Node Manager her düğümdeki kaynakları ve işlemleri takip eder.

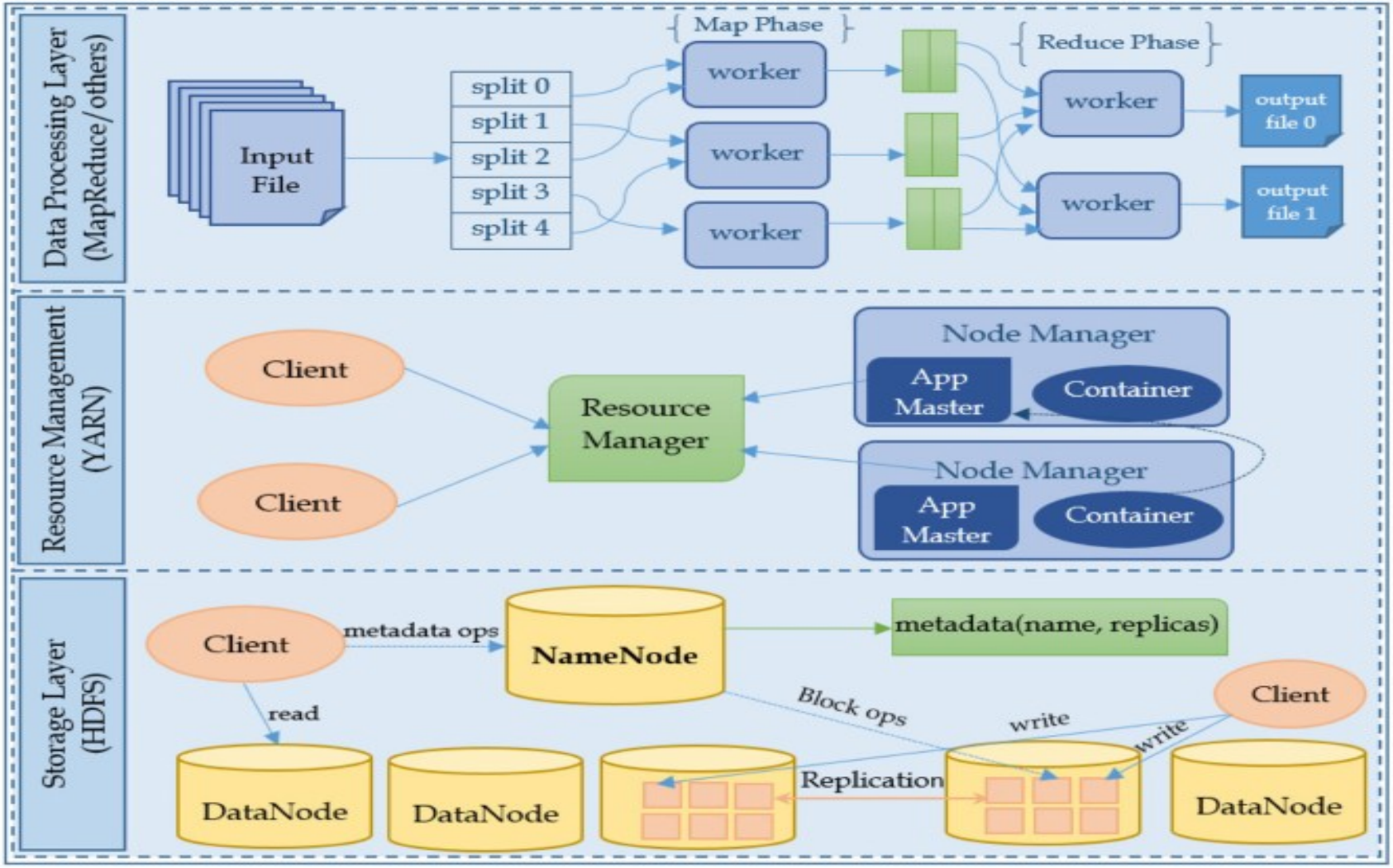


Figure 5. Apache Hadoop architecture.

4.1.2. Data Processing Model (Veri İşleme Modeli)

- Apache Hadoop, özellikle yapılandırılmamış verilerin toplu işlenmesi için kullanılır. Hadoop 1.x'te veri işleme MapReduce, depolama ise HDFS ile gerçekleştirilmiştir. Hadoop 2.x ile veri işleme ve kaynak yönetimi ayrılmış ve farklı veri işleme platformlarının kullanılması mümkün olmuştur. Buna rağmen temel veri işleme modeli MapReduce olmaya devam etmiştir.

4.1.3. Scheduling (Zamanlama)

- Hadoop'un YARN zamanlayıcısı, kümedeki kaynakları uygulamalar arasında dağıtmak için FIFO, Kapasite ve Adil (Fair) olmak üzere üç politika sunar. FIFO işleri geliş sırasına göre çalıştırırken, Kapasite her kullanıcıya minimum kaynak garantisi verir; Adil zamanlayıcı ise kaynakları çalışan işler arasında dengeli şekilde paylaştırır.

4.1.4. Fault Tolerance (Hata Toleransı)

- Hadoop, veri çoğaltma ve yedek düğümler sayesinde görev, düğüm ve kaynak yöneticisi arızalarında sistemi çalışır durumda tutarak yüksek hata toleransı sağlar.

4.2. Spark

- Spark, Hadoop MapReduce'un eksik kaldığı tekrarlı ve hızlı veri işlemlerini geliştirmek için oluşturulmuş bir büyük veri işleme çatısıdır. Verileri mümkün olduğunca RAM üzerinde (RDD) işleyerek daha hızlı çalışır ve hem toplu hem de akış verilerini işleyebilir.

4.2.1. Architecture (Spark Mimarisi)

- Spark; Spark Core, Spark SQL, Streaming, MLlib ve GraphX gibi bileşenlerden oluşur. Spark Core görevleri, bellek yönetimini ve hata kurtarmayı yönetirken, Driver görevleri dağıtır ve Worker düğümleri bu görevleri çalıştırır. Spark kendi depolama sistemine sahip değildir; HDFS, HBase ve Hive gibi sistemlerle çalışabilir.

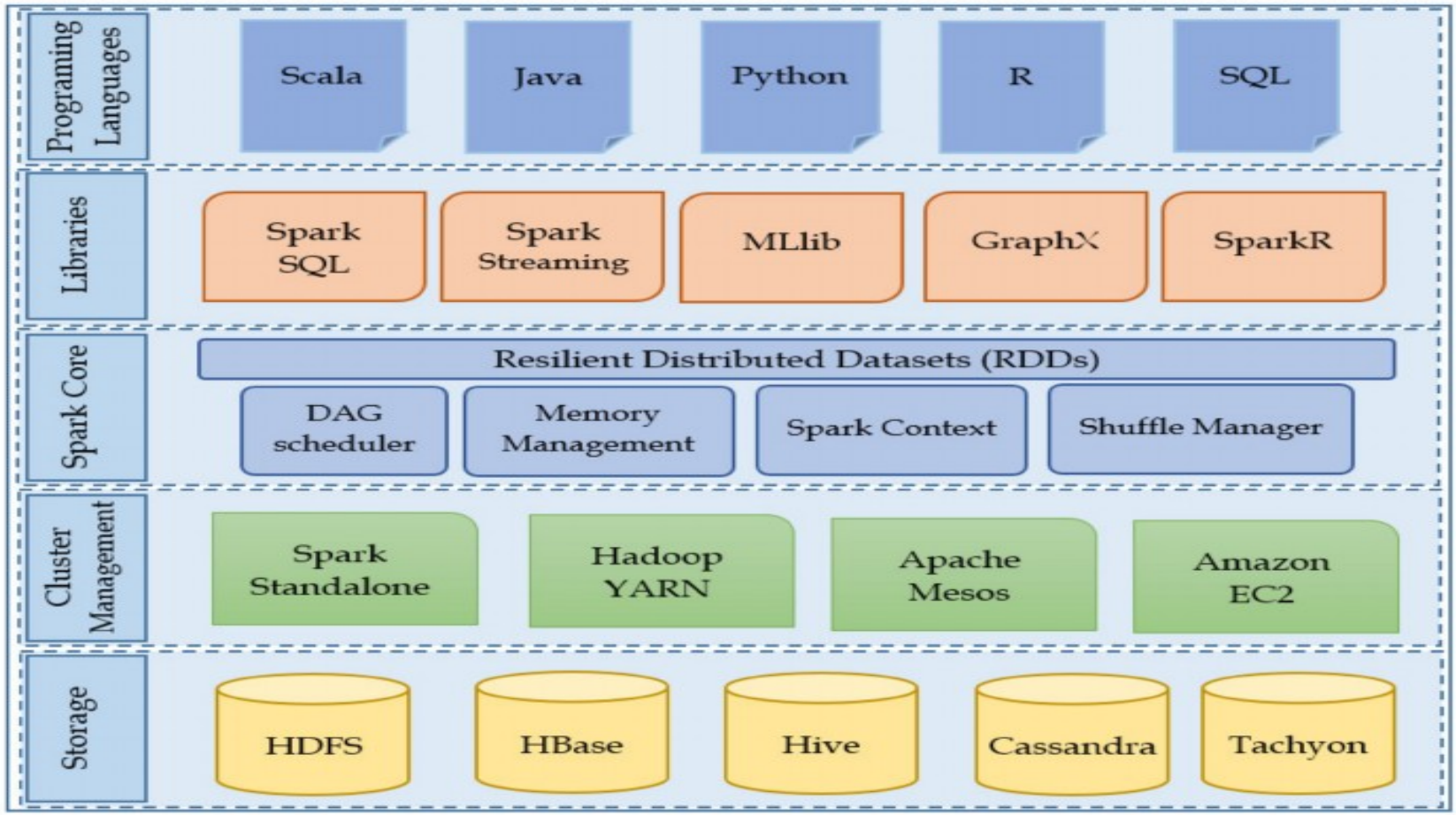


Figure 6. Layered architecture of Spark.

4.2.2. Veri İşleme Modeli

- Spark, hem toplu hem de akış verilerini işleyebilir. İşlemleri RDD yapıları üzerinde gerçekleştirir ve görevleri çalıştırmadan önce işlemleri bir DAG yapısına dönüştürür.

4.2.3. Zamanlama

- Spark'ta görevler temel olarak FIFO veya Fair zamanlayıcı ile çalıştırılır. FIFO görevleri sırayla çalıştırırken, Fair zamanlayıcı kaynakları görevler arasında daha dengeli dağıtır.

4.2.4. Hata Toleransı

- Spark, RDD'lerin geçmişini (lineage) kullanarak başarısız işlemleri yeniden çalıştırabilir. Bir düğüm arızalandığında kaybolan veriler başka bir düğümde tekrar oluşturulabilir.

4.3. Storm

- Storm, gerçek zamanlı ve sürekli akan büyük veri akışlarını düşük gecikmeyle işlemek için kullanılan dağıtık bir veri işleme çatısıdır.

4.3.1. Mimari

- Storm, Nimbus, Supervisor ve ZooKeeper bileşenlerinden oluşur; Nimbus görevleri dağıtır, Supervisor görevleri çalıştırır ve ZooKeeper koordinasyonu sağlar.

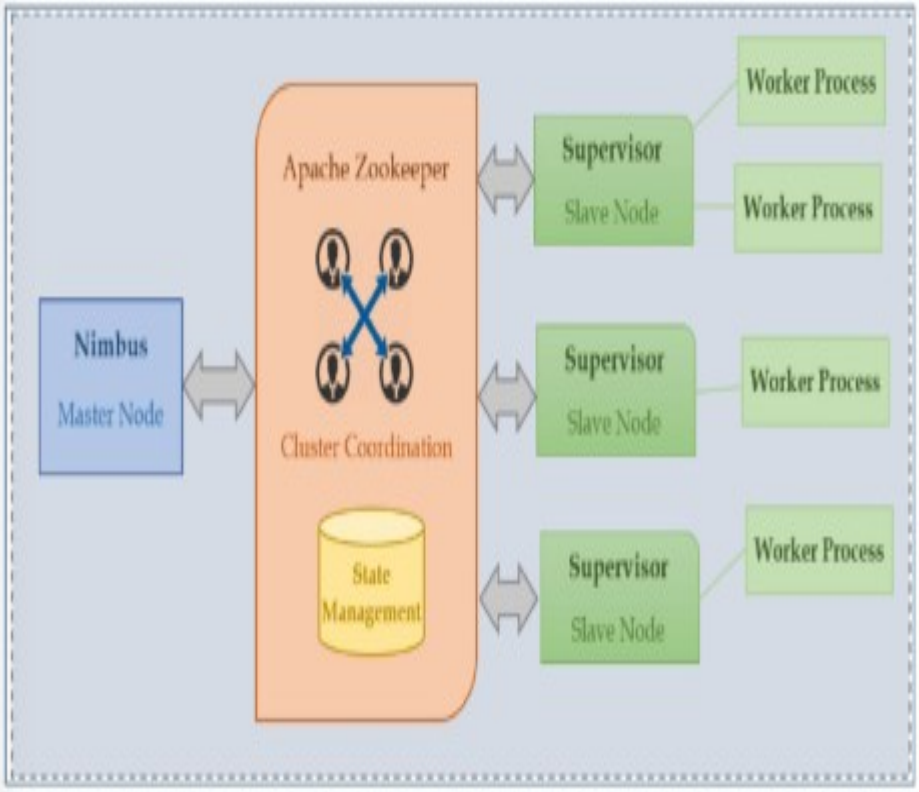


Figure 7. Apache Storm (**a**) Architecture of Apache Storm; (**b**) An illustration of Storm Topology.

4.3.2. Veri İşleme Modeli

- Storm'da veri Spout tarafından alınır, Bolt tarafından işlenir ve bu işlemler bir Topology içerisinde gerçekleştirilir.

4.3.3. Zamanlama

- Storm, görevleri kaynakları dengeli kullanacak şekilde dağıtmak için Fair, Isolation, Multitenant ve Resource Aware zamanlayıcılarını kullanır.

4.3.4. Hata Toleransı

- Bir görev veya düğüm başarısız olduğunda Storm, görevi başka bir düğüme yeniden atayarak ve gerekirse veriyi tekrar oynatarak çalışmaya devam eder.

4.4. Samza

- Samza, yüksek hacimli gerçek zamanlı veri akışlarını hızlı ve güvenilir şekilde işlemek için geliştirilmiş bir veri işleme çatısıdır.

4.4.1. Mimari

- Samza; veri akışı, çalıştırma ve veri işleme olmak üzere üç katmandan oluşur ve Kafka ile YARN gibi sistemlerle çalışabilir.

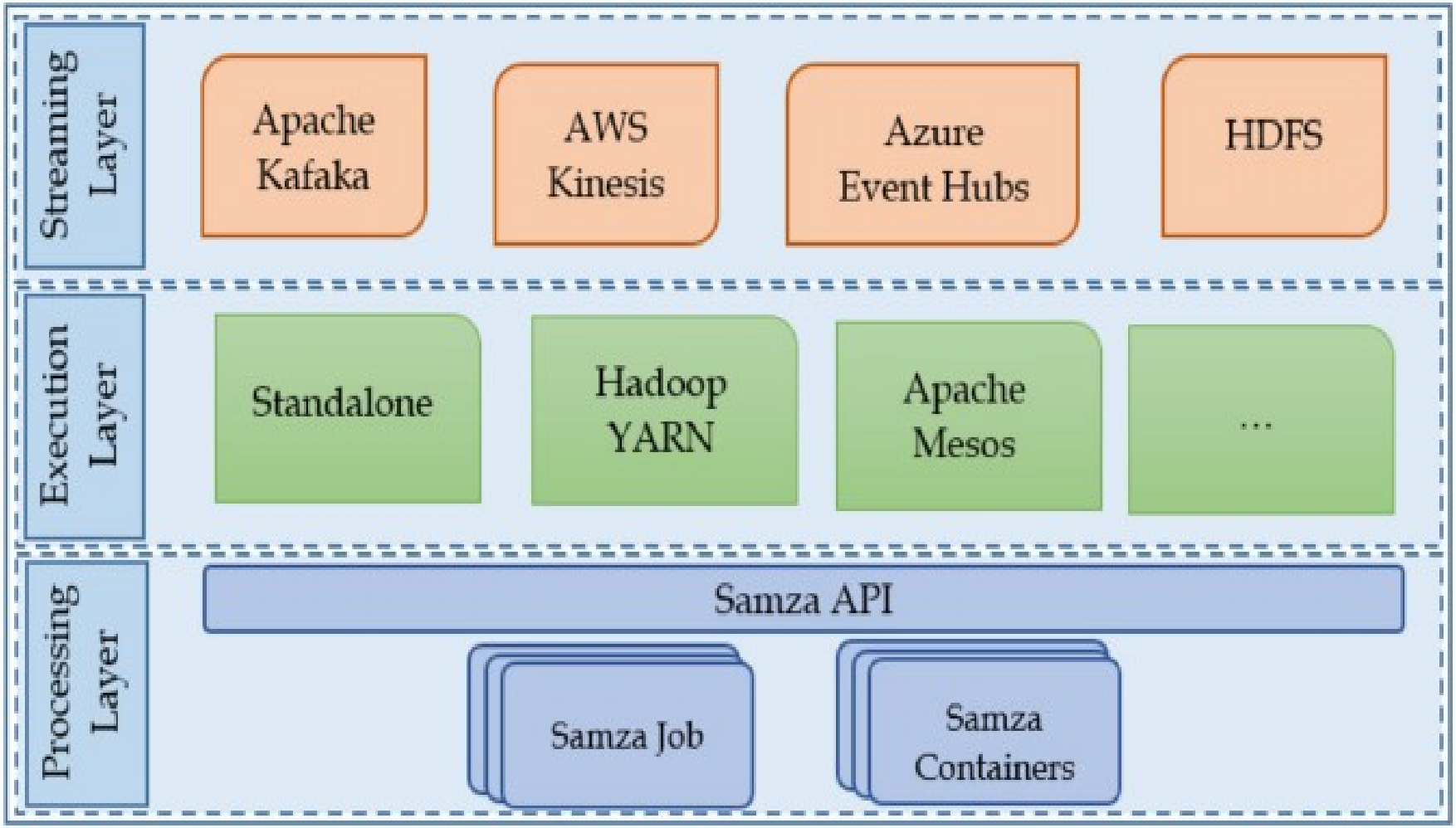


Figure 8. The layered architecture of Samza.

4.4.2. Veri İşleme Modeli

- Samza, verileri stream olarak alır ve bunları paralel çalışan job ve task'lar aracılığıyla işler.

4.4.3. Zamanlama

- Samza, görevlerin zamanlanması ve kaynakların yönetimi için YARN veya Mesos kullanır.

4.4.4. Hata Toleransı

- Samza, görevlerin durumunu changelog ile kaydeder ve hata durumunda son kaydedilen durumdan devam eder.

4.5. Flink

- Flink, hem sürekli akan hem de sınırlı verileri yüksek hız ve düşük gecikmeyle işlemek için kullanılan dağıtık bir veri işleme çatısıdır.

4.5.1. Mimari

- Flink'te Client uygulamayı gönderir, JobManager görevleri yönetir ve TaskManager verileri işler.

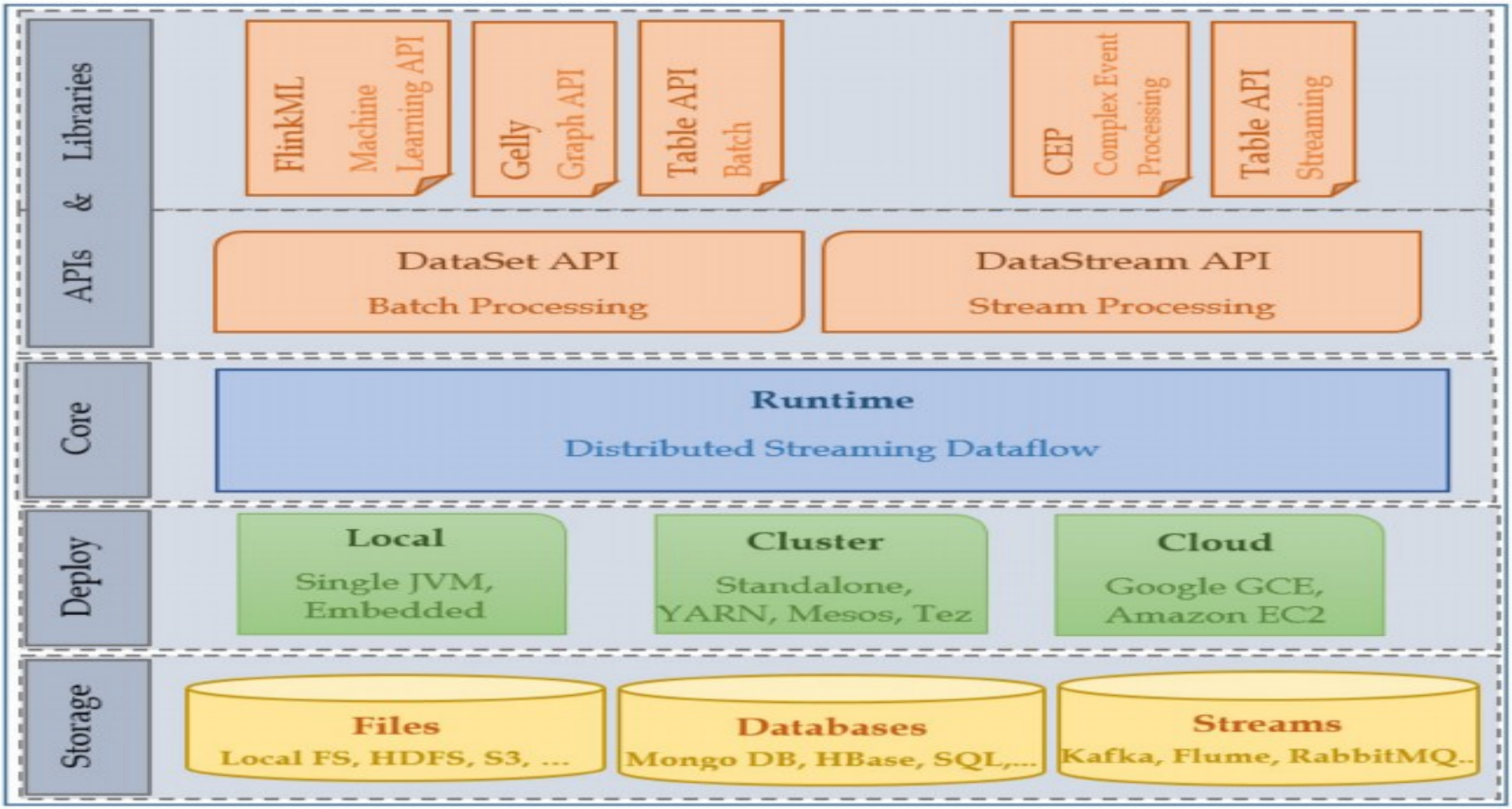


Figure 9. Apache Flink architecture.

4.5.2. Veri İşleme Modeli

- Flink, işlemleri bir veri akış grafiğine (DAG) dönüştürerek verileri paralel şekilde işler.

4.5.3. Zamanlama

- Flink, işin türüne ve kaynak durumuna göre üç farklı zamanlama yöntemi kullanarak görevleri çalıştırır.

4.5.4. Hata Toleransı

- Flink, düzenli checkpoint ve snapshot alarak hata durumunda işlemleri son güvenli noktadan yeniden başlatır ve veri kaybını önler.

5.1 Genel Özellikler

- Bu vektör; framework'ün arkasındaki kuruluşu, kullanılan programlama dilini, desteklediği dilleri ve olgunluk seviyesini karşılaştırır.

Özellik	Hadoop	Spark	Storm	Samza	Flink
Ana destekçiler	Google, Yahoo!	AMP Lab	BackType, Twitter	LinkedIn	dataArtisans
Geliştirme dili	Java	Scala	Clojure	Scala, Java	Java, Scala
Programlama dili desteği	Çoğu yüksek seviyeli dil	Herhangi bir dil	Çok geniş dil desteği	JVM dilleri	Java, Scala, Python, R
Olgunluk	Çok yüksek	Yüksek	Yüksek	Orta	Düşük

5.2 Mimari Özellikler

Özellik	Hadoop	Spark	Storm	Samza	Flink
Mimari	Master-Slave	Master-Slave	Master-Slave	Publish-Subscribe	Master-Slave
Kaynak yönetimi	YARN	YARN, Mesos	YARN, Mesos	YARN, Mesos	YARN, Mesos
Depolama	HDFS	HDFS, HBase, Hive, Cassandra	HDFS	HDFS	HDFS
Zamanlama	FIFO, Fair, Capacity	FIFO, Fair	Fair, Isolation, Multitenant, Resource Aware	YARN	Eager, Lazy, Pipelined Region
Güvenlik	Kerberos	Parola, ACL	Kerberos	Yerleşik güvenlik yok	Kerberos, TLS/SSL

5.3 Veri İşleme Özellikleri

Özellik	Hadoop	Spark	Storm	Samza	Flink
İşleme modeli	Sadece toplu	Toplu + Akış	Sadece akış	Sadece akış	Toplu + Akış
Yürütme modeli	MapReduce	DAG	Topology	DAG of Operators	Data Flow Graph
İşleme yöntemi	Batch	Micro-batching	Micro-batching	Native Streaming	Native Streaming
Bellek içi işleme	Hayır	Evet	Evet	Evet	Evet

5.4 Performans Özellikleri

- Bu vektörde üç temel performans ölçütü vardır: gecikme (latency), throughput ve ölçeklenebilirlik.

Özellik	Hadoop	Spark	Storm	Samza	Flink
Gecikme	Yüksek (saniyeler)	Düşük (birkaç saniye)	Çok düşük (saniye altı)	Çok düşük (saniye altı)	Çok düşük (saniye altı)
Throughput	Yüksek	Yüksek	Orta	Yüksek	Yüksek
Ölçeklenebilirlik	Yüksek	Orta	Orta	Düşük	Yüksek

5.5 Hata Toleransı

Framework	Hata Tespiti	Hata Kurtarma	Yüksek Kullanılabilirlik
Hadoop	Heartbeat	Veri çoğaltma	Yedek NameNode ve ResourceManager
Spark	Heartbeat	RDD Lineage	YARN üzerinden
Storm	Heartbeat	Tuple yeniden gönderme, ZooKeeper	Yedek Nimbus
Samza	Heartbeat	Changelog	YARN üzerinden
Flink	Heartbeat	Dağıtık Snapshot / Checkpoint	ZooKeeper ile yedek JobManager

5.6 Makine Öğrenmesi Desteği

Framework	Yerleşik ML Desteği	ML Uyumluluğu	Desteklenen Temel Algoritmalar
Hadoop	Hayır	Mahout, H2O, Weka, Oryx	Kümeleme, Sınıflandırma, Regresyon
Spark	MLlib	H2O, Mahout, Oryx	Kümeleme, Sınıflandırma, Regresyon, Collaborative Filtering
Storm	Hayır	SAMOA, H2O	Kümeleme, Sınıflandırma, Regresyon
Samza	Hayır	SAMOA	Sınıflandırma, Regresyon
Flink	FlinkML	SAMOA	Kümeleme, Sınıflandırma, Regresyon

5.7 Akış İşleme Özellikleri

Özellik	Spark	Storm	Samza	Flink
İşleme garantisi	Exactly-once	At-least-once	At-least-once	Exactly-once
Veri kaynakları	HDFS, DBMS, Kafka	Spout	Kafka	HDFS, DBMS, Kafka
İşleme biçimi	Micro-batch	Micro-batch	Sürekli akış	Sürekli akış
Stream yapısı	DStream	Tuple	Message	DataStream
Durum yönetimi	Stateful	Stateless	Stateful Operators	Stateful Operators

5.7 Feature Vector'ın Genel Özeti

Feature Vector	Ne inceleniyor?
Genel Özellikler	Destekçiler, olgunluk, programlama dilleri
Mimari Özellikler	Mimari, depolama, kaynak yönetimi, zamanlama, güvenlik
Veri İşleme	Toplu/akış işleme, DAG, MapReduce, bellek içi işlem
Performans	Gecikme, throughput, ölçeklenebilirlik
Hata Toleransı	Hata tespiti, kurtarma, yüksek kullanılabilirlik
Makine Öğrenmesi	ML kütüphaneleri ve algoritma desteği
Akış İşleme	İşleme garantisi, veri kaynakları, durum yönetimi

7. Sonuc

- Hadoop: Toplu veri işleme ve yüksek ölçeklenebilirlik
- Spark: Hızlı toplu + akış işleme ve güçlü ML desteği
- Storm: Gerçek zamanlı, düşük gecikmeli akış işleme
- Samza: Yüksek hacimli gerçek zamanlı veri akışları
- Flink: Düşük gecikmeli toplu + sürekli akış işleme ve güçlü hata toleransı

Dinlediğiniz için teşekkür ederim…